OBSERVABILITY

Observability

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

The Prometheus + Grafana stack lives in `docker-compose.quality.yml` under the `observability` profile. It is **opt-in** and does not run as part of the product's two-container topology (constitution Principle I — see `QUALITY_STACK.md` for the rationale).

Stack

```
prometheus          image: prom/prometheus:latest
                    profile: observability
                    port: 127.0.0.1:9090 (UI + scrape)
                    retention: 7 days
                    config: observability/prometheus.yml

grafana             image: grafana/grafana-oss:latest
                    profile: observability
                    port: 127.0.0.1:3000
                    dashboards: observability/dashboards/
                    datasources: observability/datasources/
                    default auth: admin / ${GRAFANA_PASSWORD:-
admin}
```

Both bind to `127.0.0.1` only — not exposed to the LAN by default.

Starting the stack

```
# Bring the stack up alongside the product
$COMPOSE_CMD -f docker-compose.yml -f docker-compose.quality.yml \
             --profile observability up -d

# Stop only the observability services (product stays running)
```

```
$COMPOSE_CMD -f docker-compose.quality.yml \
             --profile observability down
```

`$COMPOSE_CMD` is the runtime-detected binary (`podman compose` / `docker compose`) — see `start.sh` for the detection logic.

Viewing dashboards

1. Start the stack as above.
2. Browse to http://127.0.0.1:3000.
3. Log in with `admin` / `admin` (or the value of `GRAFANA_PASSWORD`).
4. Open the pre-provisioned **"Merge Search Service"** dashboard (UID: `qbit-merge-search`, source: `observability/dashboards/merge-search.json`).

Dashboards are auto-provisioned via `observability/dashboards/default.yml` and re-read on restart.

Metric names

The merge service exposes Prometheus metrics at `GET /metrics` on port 7187 (Phase 6 wires the scrape config — until then the Grafana dashboard charts a static sample). Names:

Metric	Type	Labels	Meaning
`qbit_merge_active_searches`	gauge	—	Searches currently running through the orchestrator
`qbit_merge_tracker_requests_total`	counter	`tracker`, `outcome`	Per-tracker request counter, outcome ∈ {success, error, timeout}
`qbit_merge_search_duration_seconds_bucket`	histogram	—	End-to-end search latency distribution
`qbit_merge_circuit_breaker_state`	gauge	`tracker`	

Metric	Type	Labels	Meaning
			Circuit-breaker state per tracker; 0=closed, 1=half-open, 2=open

The pre-provisioned dashboard panels:

1. **Active searches** (stat) — `qbit_merge_active_searches`
2. **Tracker requests per second** (timeseries) — `rate(qbit_merge_tracker_requests_total[1m])` legend `{{tracker}}`
3. **Search duration p95** (timeseries) — `histogram_quantile(0.95, sum by (le) (rate(qbit_merge_search_duration_seconds_bucket[5m])))`
4. **Circuit breakers** (stat grid) — `qbit_merge_circuit_breaker_state` grouped by `tracker`

Inspect `observability/dashboards/merge-search.json` for the full panel definitions.

Prometheus scrape config

`observability/prometheus.yml` declares the merge service as a scrape target at `localhost:7187/metrics`. Scrape interval: 15 s.

How tests assert metric existence

`tests/observability/` (pending Phase 6) asserts:

1. `GET /metrics` returns 200 with `text/plain; version=0.0.4`.
2. Every metric name in the table above appears in the response body.
3. The `/metrics` endpoint is NOT CORS-exposed (should 403 from a different origin).
4. Scraping the metric while a search is in flight shows `qbit_merge_active_searches > 0`.

Until Phase 6 lands the tests directory is empty and the observability stack runs against a stub.

OpenTelemetry

Pending. The plan introduces OpenTelemetry tracing that exports to the Grafana Tempo datasource. Status: not yet implemented.

Logging

`download-proxy/src/main.py` configures the Python stdlib logger:

```
logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s - %(levelname)s - %
        (message)s")
```

No structured logging framework today. Phase 6 target: switch to `structlog` with JSON output for ingestion by Loki.

Healthchecks

Separate from metrics but operationally relevant:

Service	Endpoint	Interval
`qbittorrent`	`curl -sf http://localhost:7185/`	30 s
`qbittorrent-proxy`	`curl -sf http://localhost:7186/`	30 s
Merge service	`GET http://localhost:7187/health`	fixture-gated
`webui-bridge.py`	`GET http://localhost:7188/health`	fixture-gated

Integration tests rely on these (see `tests/fixtures/services.py`).

Gotchas

- Prometheus retention is **7 days**. Longer history requires an upstream like Thanos or Mimir.
- Grafana dashboards read from `./observability/dashboards/` — edit the JSON in-place and restart Grafana to pick up changes.

- If `GRAFANA_PASSWORD` / `SONAR_DB_PASSWORD` / `SNYK_TOKEN` are unset, the quality compose file still comes up but the relevant service either uses its default (“admin”) or skips (snyk container exits 0).
- Tests that assert `qbit_merge_active_searches > 0` must hold a reference to the in-flight search so it does not complete before the assertion.